CLASIFICACIÓN DE RESEÑAS IMDB
CON DEEP LEARNING

Integrantes : Bryan Silva, Ignacio Manriquez, Matias Medina

CASO DE ESTUDIO

Objetivo

Nuestro objetivo es desarrollar un modelo de red neuronal el cual será entrenado con el dataset de reseñas de películas de IMDB que nos permitirá predecir los sentimientos de los usuarios al momento de generar una reseñas de forma automática clasificándolo si es (Positivo/Negativo).

LOS DATOS Y LA LIMPIEZA

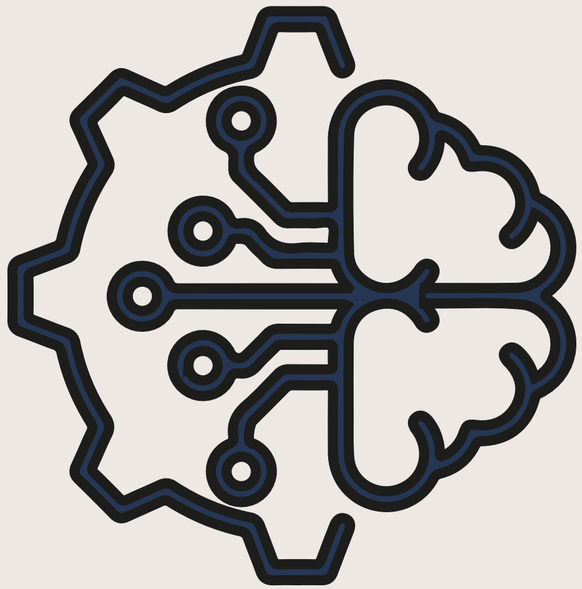
La limpieza de datos esta enfocada en nuestro archivo .csv el cual contiene un total de 50.000 reseñas de forma equilibrada 25.000 positivas y Negativas

PUNTOS CLAVE:

1 **Problema:**
Los datos se encuentran sucio con una etiqueta HTML (
/)

2 **Solución:**
Se utilizo RegEx para eliminar el ruido (HTML) junto con la función .map el cual nos ayudo a transformar las reseñas positiva(1) y negativas(0)

3 **Resultado:**
Con estos datos ya limpios de ruido y transformados en Binario estan listo para ser utilizados en la vectorizacion



PREPROCESAMIENTO CON **TF-IDF**

¿Cómo representamos el lenguaje humano en una matriz numérica sin perder la importancia de las palabras?

- **Técnica utilizada:** Term Frequency - Inverse Document Frequency (TF-IDF).

Problema: Despues de limpiar nuestros datos nos enfrentamos al problema a que nuestra red neuronal no sabe leer texto.

La Solucion (TF-IDF): Usar el Algoritmo que evalua la precuencia y la importancia de una palabra dentro de todas las reseñas de los usuarios

Limitante: Nuestro limitante es que solo podemos Preprocesar 5000 palabras relevantes de nuestro Dataset.

Impacto Positivo : Evita el colapso de la memoria RAM y reduce el riesgo en la memorización**.**

DISEÑO DE LA ARQUITECTURA DE LA RED NEURONAL

Se diseño 4 capas al cual forma una red neuronal tipo embudo el cual nos ayudo en la compresion de datos

Capa 1 | Dense (32): nuestra capa 1 verifica la informacion de las 5000 palabras al cual cada uno de las neuronas hace una conexion lo que genera un total de 160.000 parametros.

Capa 2 | Dropout: Ajustamos un dropout al 0.5 lo que nos previene de generar un overfitting y manteniendo un equilibrio entre rendimiento y aprendizaje.

Capa 3 | Dense (16): Con los hallazgos generados en la primera capa reconstruimos para entender el sentido semantico.

Capa 4 | Dense 1 (Juez): Luego de la reconstruccion de los hallazgos la capa 4 se encarga de asignar un valor a las reseñas estudiadas si la reseña es positiva la deja pasar y si es negativa le asigna un 0.

Problemas con Sobreajuste: Si nuestro dropout es ajustado al **0.2** nos generara que nuestro modelo sea torpe en aprender nuevos datos o ajustes lo que genera un **Overfitting** en cambio si nuestro dropout es ajustado al 0.8 nos generara que nuestro modelo no sea capas de procesar y aprender adecuadamente que incluso no complete la tarea adecuadamente esto lo catalogamos como **Underfitting**

FLUJO DE APRENDIZAJE

una vez construida nuestra red con sus respectivas capas nos pasamos a la fase de entrenamiento utilizando los 40.000 datos que separamos al inicio

Modelo 1 (Propuesto) Adam + Dropout 0.5

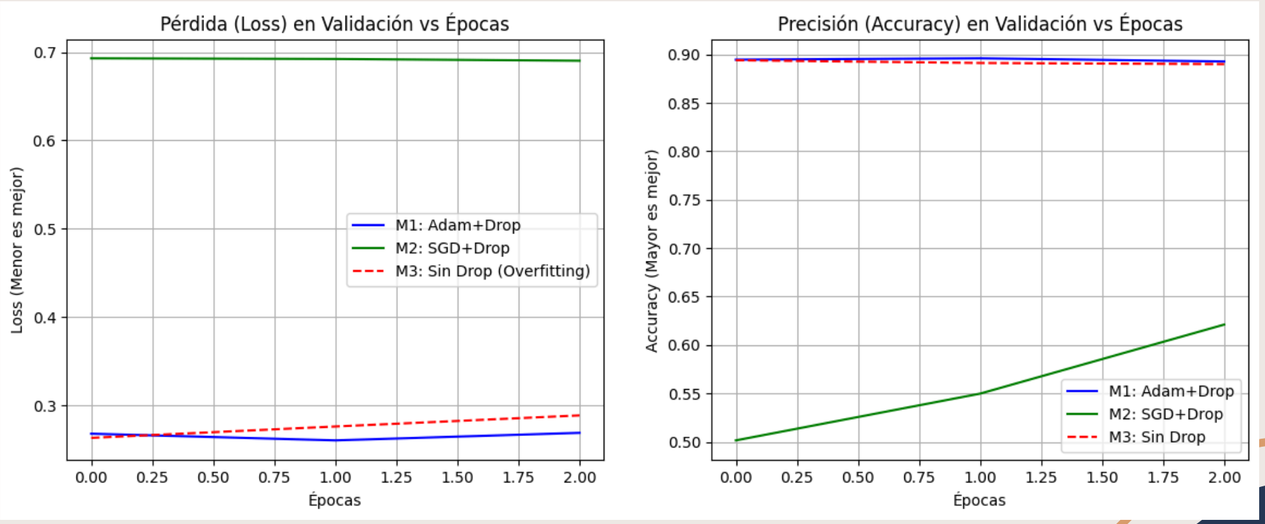
Resultado : nuestro modelo obtuvo un equilibrio en cuanto a velocidad y precision. una convergencia estable en 15 ciclos de procesamiento

Modelo 2 (Comparativo) SGD + Dropout 0.5

Resultado : los ajustes de este modelo a pesar de incluir un dropout al 0.5 su aprendizaje es lento e ineficiente el optimizador estatico no logra obtener los resultados esperados y no logra minimizar el riesgo al mismo tiempo de procesamiento.

Modelo 3 (Comparativa) Adam + Sin Dropout

Resultado : este modelo de adam al no poseer un dropout sufre de un sobreajuste severo (Overfitting) el modelo aprende los datos de entrenamiento pero es ineficiente de aprender algo nuevo que no sea lo aprendido no es adaptable



SGD = descenso de gradiente estocástico

Adam = Adaptive Moment Estimation

COMPARATIVA DE MODELOS

Para empezar no solo nos quedamos con nuestro modelo nos pusimos a comparar nuestros ajustes con otros modelos y configuraciones y de paso saber si nuestro modelo era eficiente en comparacion con el resto de diseños

Procesamiento de Datos: nuestro primer paso sera procesar 40.000 reseñas de entrenamiento dividas en bloques (Batch_size:64)

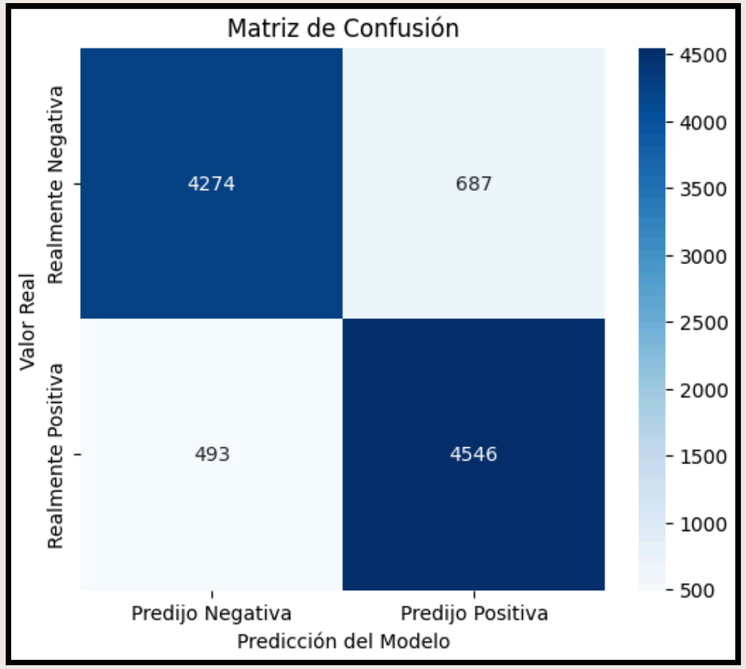
Funcion Perdida: Con binary crossintropy evualuamos el margen de error de cada prediccion.

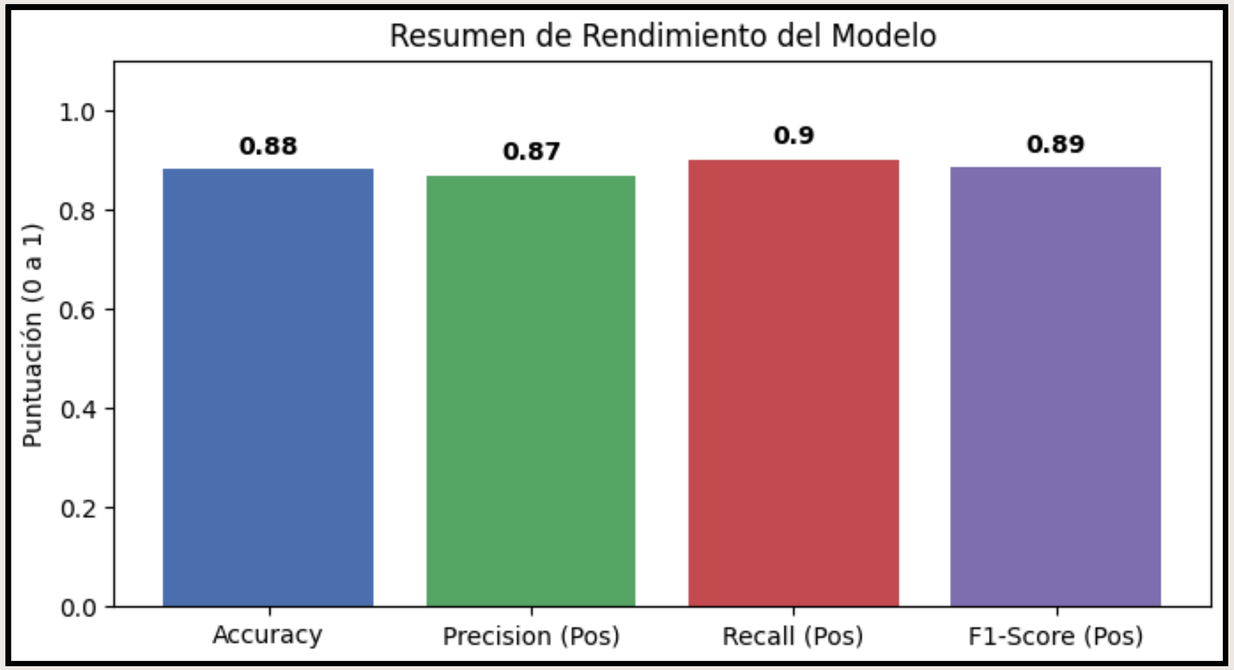
Optimizador: Adam nos ayudara a optimizar la prediccion si ocurre algun error en ello adam tiene un ajuste automatizado que ajusta nuestros 160.000 parametros para generar un aprendizaje optimo.

Ciclo Epoch (Epocas): Todo lo anterior se ajustara para que solo utilizemos 15 Epoch osea 15 ciclos de entrenamiento el cual nuestra red procesara los 40.000 reseñas o datos obtenidos.

RESULTADOS OBTENIDOS (TEST)

una vez visto la superioridad de nuestra arquitectura Adam + Dropout (0.5), procedemos a la evaluación final utilizando el set de datos de prueba, esta esta compuesta por 10.000 reseñas que el modelo anteriormente no ha procesado.



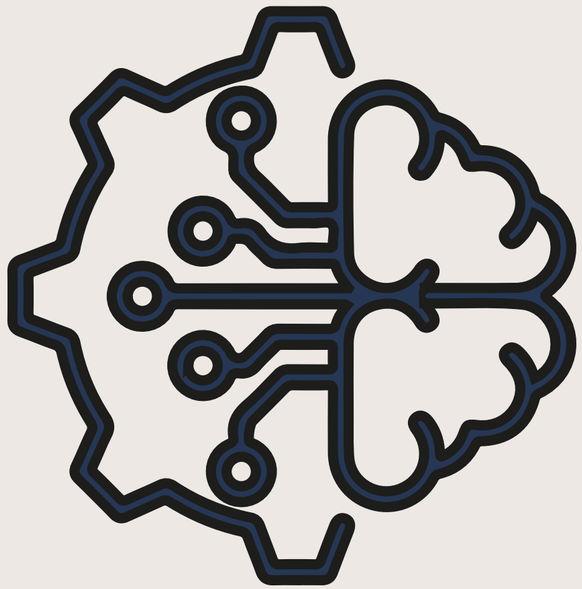


Si observamos la matriz de Confusión, el modelo logro identificar correctamente mas de 4.200 reseñas negativas y mas de 4.500 reseñas positivas al cual solo obtuvimos un margen de error pequeño con la cantidad apenas de 493 falsos negativos y 687 falsos positivos.

Bueno en cuanto a nuestro grafico de barras como podemos ver alcanzamos un accuracy solido del 88% sin embargo la metrica que mas destaco fue nuestro f1-score con un total de 89%. este indicador nos confirma que el sistema es altamente estable y no se sesga hacia un solo tipo de respuestas.

CONCLUSION FINAL

En este proyecto logramos construir un modelo de clasificación de texto basado en una red neuronal multicapa, donde demostramos que el uso de TF-IDF, junto con Adam y Dropout, permite obtener un buen equilibrio entre aprendizaje y generalización.
Los resultados alcanzados confirman que el modelo es preciso, estable y capaz de evitar el sobreajuste, cumpliendo con los objetivos planteados.



Adam = Adaptive Moment Estimation

TF-IDF = Term Frequency - Inverse Document Frequency